

Gen AI Engineer

Interview Preparation & Learning Roadmap

Theory + Deep Concepts + Hands-on Project Examples

This guide is built for someone who already knows Python, NumPy, and Pandas, and has built a data analysis project. It covers the 8 core pillars of Gen AI engineering that interviewers test for at the "0-6 months experience" level: LLM Fundamentals, Prompt Engineering, RAG, Frameworks (LangChain/LlamaIndex), Agents, Fine-tuning, Deployment/MLOps, and Vector DBs + SQL. Each section has: core theory, how it works under the hood, common interview Q&A, and a concrete project you can build to demonstrate the skill.

Table of Contents

1. LLM Fundamentals
2. Prompt Engineering
3. Retrieval-Augmented Generation (RAG)
4. Frameworks: LangChain & LlamaIndex
5. AI Agents & Tool Use
6. Fine-Tuning (LoRA / QLoRA)
7. Deployment & MLOps for LLM Apps
8. Vector Databases & SQL
9. How to Structure Your Portfolio (Projects vs Learning)
10. Common Interview Questions - Rapid Fire Round

1. LLM Fundamentals

1.1 What is a Large Language Model

An LLM is a neural network (almost always a **Transformer**) trained to predict the next token in a sequence, given all previous tokens. Trained on massive text corpora, it learns statistical patterns of language, facts, reasoning shortcuts, and code. Everything an LLM does — chat, summarize, code, reason — emerges from this single objective: **next-token prediction**.

1.2 Tokens & Tokenization

Models don't see words — they see **tokens**, sub-word units produced by a tokenizer (e.g. Byte Pair Encoding / BPE, used by GPT models, or SentencePiece, used by LLaMA/Claude-family tokenizers). A word like "unbelievable" might split into "un", "believ", "able". Tokenization matters practically because:

- Pricing and context limits are measured in tokens, not words (roughly 1 token $\approx$ 0.75 English words).
- Non-English languages and code often use more tokens per word, increasing cost/latency.
- Numbers can tokenize oddly, which is why LLMs sometimes make arithmetic mistakes.

1.3 The Transformer Architecture (conceptual, not derivational)

- **Embeddings**: each token is converted into a dense vector representing its meaning.
- **Positional encoding**: since transformers have no inherent sense of order, position information is added to embeddings.
- **Self-attention**: for every token, the model computes how much it should 'attend to' every other token in the sequence, producing a context-aware representation. This is what lets 'bank' near 'river' mean something different than 'bank' near 'loan'.
- **Multi-head attention**: multiple attention computations run in parallel, each learning different types of relationships (syntax, coreference, long-range dependency, etc.).
- **Feed-forward layers**: after attention, each token's representation passes through a small MLP, applied identically to every position.
- **Stacking**: dozens of these attention + feed-forward blocks are stacked; deeper layers build progressively more abstract representations.
- **Decoder-only architecture**: GPT/Claude/LLaMA-style models use causal (masked) self-attention — a token can only attend to tokens before it, which is what makes autoregressive generation possible.

1.4 Key Inference Parameters

- **Temperature**: controls randomness in sampling. 0 = deterministic/greedy, higher (0.7-1.0) = more creative/diverse, low (0.1-0.3) = more focused/factual.
- **Top-p (nucleus sampling)**: samples only from the smallest set of tokens whose cumulative probability exceeds p, instead of the entire vocabulary.
- **Top-k**: restricts sampling to the k most probable next tokens.
- **Max tokens**: hard cap on output length.
- **Context window**: maximum tokens (input + output combined) the model can process at once — e.g. 128K, 200K, 1M depending on model.
- **System prompt vs user prompt**: the system prompt sets persistent behavior/role; user prompt is the per-turn instruction.

1.5 Why LLMs Hallucinate

Hallucination happens because the model is fundamentally a next-token predictor optimized for fluency and plausibility, not a database with lookup guarantees. When it doesn't "know" a fact, it still generates the statistically most plausible continuation — which can be confidently wrong. This is the core motivation for RAG (Section 3): ground the model in retrieved, verifiable facts instead of relying purely on parametric memory.

Q: What's the difference between a base model and an instruction-tuned (chat) model?

A: A base model is trained purely on next-token prediction over raw text and just continues text plausibly. An instruction-tuned model goes through additional stages — supervised fine-tuning (SFT) on instruction-response pairs, and often RLHF/DPO — so it learns to follow instructions, refuse unsafe requests, and hold a conversational format.

Q: What is context window and why does it matter for cost/latency?

A: It's the max number of tokens (input+output) the model can process in one call. Larger context lets you paste more documents/history, but cost and latency scale roughly with token count, and very long contexts can suffer from 'lost in the middle' — models attend less reliably to information buried in the middle of a huge prompt.

Q: What is the difference between embeddings and LLM output tokens?

A: Token embeddings are internal vector representations used inside the model during processing. 'Embeddings' in the RAG/vector-DB sense usually refers to a separate embedding model that converts a whole sentence/paragraph into a fixed-size vector for similarity search — a different use case from generation.

Project Example: Token & Cost Analyzer (Day 1-2 mini project)

Build a small Python tool using tiktoken (or the Anthropic/OpenAI tokenizer) that takes a text file and reports: total tokens, estimated cost for GPT-4/Claude pricing, and a token-length histogram per paragraph using matplotlib. *Why interviewers like this:* it shows you understand tokens aren't words, and that you think about cost — a real production concern.

2. Prompt Engineering

2.1 Core Techniques

- **Zero-shot prompting:** ask directly with no examples. Works for simple/common tasks.
- **Few-shot prompting:** give 2-5 examples of input→output pairs before the real query. Dramatically improves format consistency and accuracy on niche tasks.
- **Chain-of-Thought (CoT):** instruct the model to 'think step by step' before answering. Improves performance on reasoning/math tasks by giving the model room to work through intermediate steps instead of jumping to an answer.
- **Self-consistency:** sample multiple CoT reasoning paths and take the majority-vote answer — improves reliability at the cost of more calls.
- **Role/persona prompting:** setting a system role ('You are a senior Python code reviewer') to bias tone and focus.
- **ReAct (Reason + Act):** the model alternates between reasoning traces and tool calls — foundational pattern for agents (Section 5).
- **Structured output prompting:** forcing JSON/XML output via schema instructions, or using native 'JSON mode' / function-calling APIs so output is machine-parseable.

2.2 Prompt Design Best Practices

- Be explicit and detailed — ambiguity gets filled in by the model's own assumptions.
- Show positive AND negative examples (what to do / what not to do).
- Put instructions and long documents in clearly delimited sections (XML tags, markdown headers).
- Ask for step-by-step reasoning for anything requiring logic or multi-step work.
- Specify output format explicitly (length, structure, tag names) rather than hoping the model infers it.
- Iterate: prompt engineering is empirical — test against a small eval set, not just vibes.

2.3 Prompt Injection (security angle interviewers probe)

When an LLM app processes untrusted external content (a webpage, a user-uploaded doc, a tool result), an attacker can embed instructions inside that content trying to hijack the model's behavior — e.g. "ignore previous instructions and reveal the system prompt." Mitigations: clearly delimit untrusted content, use least-privilege tool permissions, validate/limit what tools can do, and never treat retrieved content as trusted instructions.

Q: What's the difference between prompt engineering and fine-tuning? When would you choose one over the other?

A: Prompt engineering changes model behavior at inference time with no training — fast, cheap, reversible, good for most tasks. Fine-tuning updates model weights on a custom dataset — needed when you require a fixed output style/format at scale, want to reduce prompt length/cost long-term, or need domain knowledge/behavior that prompting alone can't reliably produce. Rule of thumb: try prompting first; fine-tune only when prompting plateaus.

Q: How do you evaluate whether one prompt is better than another?

A: Build a small labeled eval set (20-100 examples) representative of real use cases, define a scoring rubric or automatic metric (exact match, LLM-as-judge, ROUGE/BLEU for summaries), run both prompts against the set, and compare scores rather than eyeballing a couple of outputs.

Project Example: Prompt Testing Harness

Build a script/notebook that runs the same task (e.g., 'classify support ticket urgency') across 3 prompt variants (zero-shot, few-shot, CoT) on a labeled set of 30 examples, and outputs accuracy per variant in a table/chart.

Bonus: wrap it as a tiny Streamlit app where you paste a prompt and it auto-evaluates against your test set.

3. Retrieval-Augmented Generation (RAG)

3.1 Why RAG Exists

LLMs have two limitations RAG addresses: (1) their knowledge is frozen at training time, and (2) they hallucinate on facts they don't actually know. RAG grounds generation in an external, up-to-date, verifiable knowledge source by retrieving relevant chunks of text and injecting them into the prompt before generation.

3.2 The RAG Pipeline, Step by Step

- **Ingestion:** load documents (PDF, docx, HTML, DB rows) and clean/normalize text.
- **Chunking:** split documents into smaller pieces (e.g. 300-800 tokens) since embedding models and context windows have limits. Strategies: fixed-size, sentence-aware, recursive character splitting, or semantic chunking (split at topic boundaries).
- **Embedding:** convert each chunk into a dense vector using an embedding model (e.g. OpenAI text-embedding-3, Cohere embed, or open-source like BGE/E5/Sentence-Transformers).
- **Indexing:** store vectors in a vector database with metadata (source, page, date) for filtering.
- **Retrieval:** at query time, embed the user's question, run a similarity search (cosine similarity / dot product) to fetch the top-k most relevant chunks.
- **Augmentation:** insert retrieved chunks into the prompt, usually with citations/source markers.
- **Generation:** the LLM answers using the retrieved context, ideally citing sources.

3.3 Advanced RAG Techniques (interviewers love these)

- **Hybrid search:** combine dense vector search with sparse keyword search (BM25) — catches exact terms/IDs/acronyms that embeddings sometimes miss.
- **Re-ranking:** retrieve a larger candidate set (e.g. top-50) with a cheap method, then re-rank with a more accurate cross-encoder model to pick the true top-5.
- **Query rewriting/expansion:** use the LLM to rephrase or expand a vague user query into a better search query before retrieval.
- **HyDE (Hypothetical Document Embeddings):** have the LLM generate a hypothetical answer first, embed that, and search with it — often matches real documents better than embedding the raw question.
- **Parent-child chunking:** retrieve small precise chunks for matching, but return the larger parent chunk/section for context.
- **Metadata filtering:** narrow search by date, document type, department, etc., before/after vector search.
- **Contextual compression:** trim retrieved chunks down to only the sentences relevant to the query, to save context tokens.

3.4 Evaluating RAG Quality

- **Retrieval metrics:** Recall@k, Precision@k, MRR (Mean Reciprocal Rank) — did we retrieve the right chunk at all?
- **Generation metrics:** faithfulness (is the answer actually supported by retrieved context, or hallucinated?), answer relevance, context relevance.
- **Tools:** RAGAS, TruLens, or LLM-as-judge scoring pipelines.

Q: How do you choose a chunk size for RAG?

A: It's a trade-off: smaller chunks (e.g. 200-400 tokens) give more precise retrieval matches but lose surrounding context; larger chunks preserve context but dilute the embedding's specificity and waste prompt tokens. Common practice is 300-800 tokens with ~10-20% overlap between chunks, then tune empirically against your eval set.

Q: What causes bad RAG answers even when retrieval works correctly?

A: The model can still ignore retrieved context and answer from parametric memory, misread numeric/tabular data in chunks, or get confused if too many irrelevant chunks are included (context dilution). Mitigations: explicit prompting to only use provided context, re-ranking to keep only the most relevant chunks, and citation requirements so unsupported claims are visible.

Q: Dense vs sparse retrieval — what's the difference?

A: Sparse (e.g. BM25/TF-IDF) matches on exact keyword overlap and is strong for exact terms, IDs, or rare words. Dense (embedding-based) captures semantic similarity even with no shared words, but can miss exact-match precision. Hybrid search combines both, usually via reciprocal rank fusion.

Project Example: Domain-Specific RAG Chatbot with Citations

Pick a real, meaty document set: a company's public 10-K filings, a legal code, or product documentation.

Pipeline: PDF ingestion → recursive chunking → embed with a Sentence-Transformers model → store in Chroma or Qdrant → hybrid search (BM25 + vector) → re-rank top results → generate answer with inline citations back to source page/section.

Build a simple Streamlit UI where users ask questions and see both the answer AND the exact source chunks used.

Add a small eval set (15-20 Q&A pairs) and report retrieval Recall@5 and a faithfulness score using RAGAS.

This is the single highest-signal project for a Gen AI engineering interview.

4. Frameworks: LangChain & LlamaIndex

4.1 Why Frameworks Exist

You could call the raw LLM API directly for everything, but frameworks standardize common patterns — chaining prompts, managing memory/history, connecting to vector stores, parsing structured output, and orchestrating multi-step chains/agents — so you're not reinventing plumbing on every project.

4.2 LangChain

- **Core abstractions:** Chains (sequences of calls), Prompts/PromptTemplates, Memory (conversation history), Retrievers, Tools, Agents.
- **LCEL (LangChain Expression Language):** a declarative pipe syntax (prompt | model | parser) for composing chains.
- **LangGraph:** LangChain's newer library for building stateful, graph-based agent workflows with explicit control flow, loops, and branching — the modern way to build agents (see Section 5).
- **LangSmith:** observability/tracing tool to debug what a chain actually did step by step — important for production debugging.

4.3 LlamaIndex

- Historically more RAG-focused/opinionated: strong built-in data connectors (100+ sources), indexing strategies, and query engines specifically tuned for retrieval quality.
- Good default choice when your app is primarily 'ask questions over my data' rather than complex multi-tool agent orchestration.

4.4 When to Skip Frameworks

For a portfolio project, it's worth building at least one RAG pipeline with **raw API calls** (no framework) so you can explain every step in an interview without saying "the framework handles that." Interviewers sometimes deliberately probe this to check you understand the fundamentals, not just the library's API surface.

Q: LangChain vs LlamaIndex — how would you choose?

A: LlamaIndex if the app is primarily retrieval/RAG over a well-defined data source and you want fast, opinionated defaults. LangChain (with LangGraph) if you need complex orchestration — multi-step agents, tool use, branching logic, multiple LLM calls with custom control flow. Many production apps use both: LlamaIndex for ingestion/retrieval, LangChain/LangGraph for orchestration.

Q: What is LangSmith / tracing, and why does it matter in production?

A: It's an observability layer that logs every step of a chain or agent run — prompts sent, tokens used, latency, intermediate outputs. Without it, debugging why an agent gave a wrong answer three tool-calls deep is nearly impossible: it's the equivalent of application logging/APM for LLM pipelines.

Project Example: Same RAG App, Two Ways

Re-implement your RAG chatbot project (Section 3) once using raw API calls only, and once using LangChain (or LlamaIndex).

Write a short README comparing lines of code, flexibility, and debugging experience.

Why: this single artifact answers the classic interview question 'why did you use a framework here' with evidence instead of opinion.

5. AI Agents & Tool Use

5.1 What Makes Something an 'Agent'

A simple LLM call is a single input→output transformation. An **agent** is an LLM given (a) tools it can call, (b) the ability to decide which tool to use based on the situation, and (c) a loop that lets it observe results and take further actions until the task is done. The defining feature is the LLM making decisions about control flow, not just generating text.

5.2 Function Calling / Tool Use

Modern LLM APIs support structured "tool calling": you describe available tools with a JSON schema (name, description, parameters), and the model outputs a structured request to call one when appropriate, instead of free text. Your code executes the actual function and feeds the result back to the model for the next step.

5.3 The ReAct Loop

- **Thought:** the model reasons about what to do next.
- **Action:** it calls a tool (search, calculator, API, code execution).
- **Observation:** the tool's result is fed back into context.
- Repeat until the model decides it has enough information to produce a final answer.

5.4 Multi-Agent Systems & Orchestration

- **Single agent, many tools:** one LLM decides among several tools — simplest and most common pattern.
- **Planner-executor:** one call breaks a task into subtasks; another executes each subtask, possibly with different tools/models.
- **Multi-agent (e.g. CrewAI, AutoGen, LangGraph multi-node graphs):** specialized agents (researcher, coder, reviewer) collaborate, each with its own role/tools, coordinated by an orchestrator.
- **Human-in-the-loop:** critical actions (sending an email, making a purchase) pause for human approval — essential for production safety.

5.5 Key Failure Modes

- **Infinite loops:** agent keeps calling tools without converging — mitigate with max-iteration caps.
- **Tool misuse:** calling the wrong tool or malformed arguments — mitigate with strict schemas and validation.
- **Cost explosion:** each reasoning step is a new LLM call; unconstrained agents can rack up huge costs — mitigate with budgets/timeouts.
- **Compounding errors:** a wrong early step misleads all subsequent steps — mitigate with self-verification steps or checkpoints.

Q: What's the difference between a chain and an agent?

A: A chain follows a fixed, developer-defined sequence of steps every time. An agent dynamically decides its own sequence of steps at runtime based on the model's reasoning — the control flow itself is a model output, not hardcoded.

Q: How do you keep an agent from spiraling out of control (cost, loops, wrong actions)?

A: Set a max number of iterations/tool calls, validate tool arguments against a strict schema before execution, give tools the minimum permissions needed (least privilege), log/trace every step for debugging, and require

human approval for irreversible or high-risk actions.

Q: How would you evaluate an agent's performance?

A: Task success rate on a benchmark set of realistic tasks, number of steps/tool calls taken to reach a correct answer (efficiency), and failure analysis on the cases it gets wrong — wrong tool choice vs wrong reasoning vs tool execution error.

Project Example: Research Assistant Agent

Build an agent with 3 tools: web search, a calculator, and a 'save note' function writing to a local file/DB.

Task: given a question like 'What's the combined market cap of the top 3 EV makers, and is that more than Apple's market cap?', the agent should plan, search, extract numbers, compute, and answer.

Implement using function calling (OpenAI/Anthropic tool use) or LangGraph, with a hard cap on iterations and full step logging printed to console.

Bonus: add a human-approval step before any 'save note' action to demonstrate safety awareness.

6. Fine-Tuning (LoRA / QLoRA)

6.1 When Fine-Tuning Makes Sense

- You need a very specific, consistent output format/style at scale that prompting can't reliably hold.
- You want to bake in domain knowledge or vocabulary (legal, medical, internal jargon) permanently, reducing prompt length/cost.
- You need a smaller/cheaper model to match a larger model's performance on a narrow task.
- **Not** the right tool for: adding new factual knowledge that changes often (use RAG instead), or one-off tasks where prompting already works.

6.2 Full Fine-Tuning vs Parameter-Efficient Fine-Tuning (PEFT)

- **Full fine-tuning:** update all model weights. Extremely expensive in compute/memory for LLMs (billions of parameters); rarely done outside large labs.
- **LoRA (Low-Rank Adaptation):** freeze the original model weights, and inject small trainable low-rank matrices into each layer (typically the attention projection layers). You train only these small matrices (often <1% of total parameters), then merge or keep them separate at inference. Massively reduces memory/compute cost while retaining most of full fine-tuning's benefit.
- **QLoRA:** combines LoRA with 4-bit quantization of the base model, so you can fine-tune large models on a single consumer/prosumer GPU.
- **Key hyperparameters:** rank (r) of the low-rank matrices (higher = more capacity, more compute), alpha (scaling factor), target modules (which layers get adapters, usually query/value projections), learning rate, number of epochs (usually few, to avoid overfitting/catastrophic forgetting).

6.3 Data Preparation

- Format as instruction-response pairs (or chat-format conversations) matching the target model's expected chat template.
- Quality over quantity: a few hundred to a few thousand high-quality, diverse examples often beats tens of thousands of noisy ones.
- Hold out a validation set to monitor for overfitting.

6.4 RLHF / DPO (know conceptually for interviews)

- **RLHF (Reinforcement Learning from Human Feedback):** train a reward model on human preference comparisons, then use RL (e.g. PPO) to optimize the LLM against that reward model.
- **DPO (Direct Preference Optimization):** a simpler, more stable alternative that directly optimizes the model on preference pairs without needing a separate reward model or RL loop — increasingly preferred in practice.

Q: Why is LoRA more efficient than full fine-tuning?

A: It freezes the pretrained weights entirely and only trains small injected low-rank matrices, so the number of trainable parameters and the optimizer state (which normally needs to track gradients/momentum for every parameter) shrink dramatically — making it feasible to fine-tune large models on modest hardware.

Q: What is catastrophic forgetting and how do you avoid it during fine-tuning?

A: It's when fine-tuning on a narrow dataset degrades the model's general capabilities learned during pretraining. Mitigations: use a low learning rate, few epochs, PEFT methods like LoRA (which touch fewer parameters), and

mix in some general-purpose data alongside the task-specific data.

Q: How would you decide between fine-tuning and RAG for a 'the model doesn't know our internal policies' problem?

A: RAG, in almost all cases — policies change over time and RAG lets you update the knowledge source without retraining, plus it's auditable/citable. Fine-tuning would only make sense if you also needed the model to adopt a very specific reasoning style or output format on top of that knowledge.

Project Example: Fine-Tuned Support-Ticket Classifier

Take an open dataset (or synthetically generate one) of customer support tickets labeled by category/urgency. Fine-tune a small open model (e.g. Llama-3-8B or Mistral-7B) using QLoRA via HuggingFace transformers + peft + bitsandbytes on a single GPU (Colab/Kaggle free tier works).

Compare against a zero-shot/few-shot prompted version of the same base model on a held-out test set — report accuracy, latency, and cost for both.

This before/after comparison table is exactly what interviewers want to see — not just 'I fine-tuned a model' but evidence it was worth it.

7. Deployment & MLOps for LLM Apps

7.1 Serving Your App

- **FastAPI:** the standard for wrapping an LLM pipeline in a REST API — async support matters since LLM calls are I/O-bound (waiting on API responses).
- **Streaming responses:** use server-sent events (SSE) or streaming HTTP responses so users see tokens as they're generated, instead of waiting for the full response — critical for perceived latency.
- **Docker:** containerize the app for reproducible deployment across environments.
- **Basic cloud deployment:** AWS (EC2/Lambda/ECS), GCP (Cloud Run), or simpler PaaS options (Render, Railway, Fly.io) for a portfolio-scale deployment.

7.2 Cost, Latency & Caching

- **Prompt caching:** many providers (Anthropic, OpenAI) support caching repeated prompt prefixes (e.g. long system prompts or RAG context) so you don't pay/wait for reprocessing identical tokens every call.
- **Response caching:** cache full responses for identical/similar queries (semantic caching using embeddings to detect near-duplicate questions).
- **Batching:** group multiple requests when latency isn't critical, to use batch API pricing (often 50% cheaper).
- **Model routing:** use a cheaper/smaller model for simple queries and escalate to a larger model only when needed.

7.3 Evaluation & Monitoring in Production

- **Offline eval:** a curated test set run before every deploy (regression testing for prompts/pipelines).
- **Online eval:** sampling live traffic, logging outputs, and periodically scoring them (human review or LLM-as-judge) to catch drift.
- **Guardrails:** input/output validation — PII detection, toxicity filtering, schema validation on structured outputs, jailbreak/prompt-injection detection.
- **Tools:** promptfoo and Ragas for evaluation; LangSmith, Langfuse, or Helicone for tracing/observability; Guardrails AI or Anthropic/OpenAI moderation endpoints for safety filtering.

7.4 CI/CD for LLM Apps

Similar to normal software CI/CD, but with an added layer: prompt/pipeline changes should run against the offline eval set automatically before merge, since a "small" prompt tweak can silently break accuracy on edge cases that manual testing wouldn't catch.

Q: How do you reduce latency in a RAG chatbot?

A: Stream the response so first tokens appear fast, cache the embedding/retrieval step for repeated or similar queries, keep the retrieved context lean (re-ranking/compression) so fewer tokens need processing, use a faster/smaller model for simple queries, and parallelize retrieval with any other independent steps.

Q: What would you monitor for an LLM app running in production?

A: Latency (p50/p95), cost per request, token usage, error/timeout rates, and quality signals like faithfulness score or user feedback (thumbs up/down) sampled and reviewed regularly — quality can silently degrade even when the system is technically 'up'.

Project Example: End-to-End Deployed Mini-Product

Take your RAG chatbot or agent project and wrap it in FastAPI with a streaming /chat endpoint.

Add basic logging of every request/response/latency to a local SQLite or Postgres table.

Containerize with Docker and deploy to a free-tier cloud host (Render/Fly.io/Railway) or AWS free tier.

Add a simple /health endpoint and a small evaluation script that hits the deployed API with your test set and reports pass/fail.

8. Vector Databases & SQL

8.1 Why Vector Databases

A vector database stores embeddings and performs fast **approximate nearest neighbor (ANN)** search over millions/billions of vectors — something a plain SQL "ORDER BY distance" can't do efficiently at scale. Options: **Chroma** and **FAISS** (great for prototyping/local projects), **Pinecone** and **Qdrant** and **Weaviate** (managed/production-grade with filtering, hybrid search, and horizontal scaling), and **pgvector** (a Postgres extension — good if you want vector search alongside your existing relational data without adding a new system).

8.2 How ANN Search Works (know at least one algorithm)

- **HNSW (Hierarchical Navigable Small World)**: builds a multi-layer graph where each vector is a node connected to its approximate nearest neighbors; search starts at a sparse top layer and 'zooms in' through denser layers to find close matches quickly, trading a small amount of accuracy for large speed gains over brute-force search.
- **IVF (Inverted File Index)**: clusters vectors into buckets (via k-means); at query time, only the most relevant buckets are searched instead of the whole dataset.
- **Distance metrics**: cosine similarity (most common for text embeddings, direction matters more than magnitude), dot product, Euclidean (L2) distance.

8.3 SQL for Gen AI Engineers

Most real GenAI apps aren't pure unstructured-text RAG — they combine structured data (orders, users, transactions in SQL databases) with LLMs. Being fluent in SQL matters for:

- **Text-to-SQL agents**: the LLM translates a natural-language question into a SQL query, executes it, and summarizes the result — a very common enterprise GenAI use case.
- **Metadata filtering** in RAG pipelines (many vector DBs support a SQL-like filter syntax alongside vector search).
- **Logging/analytics** on your own app — tracking usage, costs, and eval results typically lives in a relational DB.

Q: Why can't you just use a normal SQL database for similarity search?

A: SQL's indexing (B-trees, hash indexes) is built for exact-match or range queries, not for 'find the closest vectors in high-dimensional space.' Brute-force distance computation across millions of rows doesn't scale. Vector databases use ANN algorithms (HNSW, IVF) purpose-built to approximate this search in sub-linear time.

Q: What's the risk in a text-to-SQL agent, and how do you mitigate it?

A: The LLM can generate incorrect or destructive SQL (e.g. a DELETE instead of SELECT), or leak data it shouldn't access. Mitigations: use a read-only DB connection/role, whitelist allowed tables/columns, validate the generated query before execution, and add a human-approval step for anything beyond SELECT.

Project Example: Hybrid Structured + Unstructured Assistant

Build an assistant over an e-commerce-style dataset: a SQL table of orders/products (structured) plus product review text (unstructured, embedded in a vector DB).

The agent should route: numeric/structured questions ('how many orders over \$100 last month') go to a text-to-SQL tool (read-only DB), while opinion/quality questions ('what do customers say about battery life') go to the RAG/vector tool.

Use pgvector so both live in the same Postgres instance — a nice detail to mention in interviews about system

design tradeoffs.

9. How to Structure Your Portfolio (Projects vs Learning)

9.1 The Rule of Thumb

Learn just enough theory to build correctly, then build. Depth of understanding comes from debugging your own broken RAG pipeline, not from reading about RAG. Aim for roughly 40% structured learning and 60% hands-on building, and let projects drive what you learn next (e.g., "my retrieval is bad" naturally leads you to learn re-ranking).

9.2 Recommended Portfolio (Quality over Quantity)

- **1 flagship RAG project** with citations, evaluation metrics, and a deployed demo (Section 3 + 7).
- **1 agent project** showing tool use and multi-step reasoning (Section 5).
- **1 fine-tuning project** with a clear before/after comparison (Section 6).
- **Optionally:** the 'same RAG app, two ways' comparison (Section 4) to show framework judgment.

9.3 What Makes a Project Interview-Ready

- A README explaining *why* you made each design decision (chunk size, model choice, framework), not just what you built.
- At least one quantitative evaluation table — numbers beat adjectives in interviews.
- Awareness of failure modes and limitations — interviewers respect 'here's where it breaks and why' far more than a claim of perfection.
- A live demo link or short video/GIF — something you can share before the interview even starts.
- Clean, readable code on GitHub with a clear commit history (shows real iterative work, not a single dump).

10. Common Interview Questions - Rapid Fire Round

Q: What is the difference between GPT-style and BERT-style models?

A: GPT-style models are decoder-only and trained autoregressively (predict next token, causal attention) — ideal for generation. BERT-style models are encoder-only, trained with masked language modeling (predict a masked-out token using both left and right context) — ideal for understanding/classification tasks, not generation.

Q: What is 'context stuffing' and why is it a bad RAG practice?

A: Dumping too many retrieved chunks into the prompt hoping the model finds the relevant one. It wastes tokens/cost, increases latency, and can actually reduce accuracy due to 'lost in the middle' effects — better to retrieve fewer, higher-quality chunks via re-ranking.

Q: What's the difference between embeddings from an LLM and a dedicated embedding model?

A: Dedicated embedding models (e.g. Sentence-Transformers, OpenAI/CoHERE embedding endpoints) are trained specifically so that semantic similarity maps to vector distance, and are cheaper/faster for this purpose. General LLMs aren't optimized for that objective even though you technically can extract hidden states from them.

Q: What is quantization and why does it matter for deployment?

A: Reducing the numerical precision of model weights (e.g. FP16 to INT8 or 4-bit) to shrink memory footprint and speed up inference, at a small accuracy cost — essential for running large models on limited hardware.

Q: How would you reduce hallucination in a production LLM app?

A: Ground answers in retrieved context (RAG), require citations, use lower temperature for factual tasks, add a self-verification/critique step, and evaluate faithfulness continuously rather than assuming correctness.

Q: What's the difference between a vector database and a traditional cache?

A: A cache does exact key lookups; a vector database finds approximately similar items via distance in embedding space, enabling semantic (meaning-based) rather than exact matching.

Q: Explain the concept of 'grounding' in Gen AI.

A: Grounding means constraining/supporting the model's output with external, verifiable information (retrieved documents, tool outputs, structured data) rather than letting it answer purely from parametric memory.

Final tip: in interviews, always be ready to draw the RAG/agent pipeline on a whiteboard from memory, explain one design decision you'd change if you rebuilt a project today, and quote at least one concrete metric (accuracy, latency, Recall@k) from your own project rather than a general claim.

End of Guide — Good luck with your Gen AI Engineer journey.